



Spring IoC Container

The spring IoC container takes the control / responsibility of object creation & management lifecycle away from the developer.

This helps the developer save time & focus mainly on the business logic

→ Normal Java Objects (POJO) → Beans when provided by the Spring IoC container

→ All beans are objects, but all objects are not beans

Java Reflection API

There is an API provided by Java which can store metadata (information) about a particular class itself!

→ For example how many constructors does it have, how many fields are present, what are their access modifiers etc.

```
Class <?> class1 = Student.class
```

→ Spring's IoC container internally uses the Reflections API to manage the beans.

Application Context

→ IoC container uses the Application Context to manage the beans

→ @Component: Annotation to specify Spring's IoC container to make a bean on the specified class

→ @Configuration: Specifies a configuration class which the Spring's

IOC container will use to create / manage the beans

→ `@ComponentScan("package.name")`: Specifies which package & its subpackages to scan & create beans of classes with `@Component` annotation.

Dependency Injection

→ It's the means used by the Spring's IOC container to wire all the project's dependencies together

1. Constructor Injection (Recommended / Best Practice)

```
private final PaymentService paymentService;
```

```
@Autowired
```

```
public OrderService (PaymentService paymentService) {  
    | this.paymentService = paymentService;  
}
```

→ Here, using `@Autowired` on the constructor, we are letting Spring know to automatically wire the dependency of `paymentService` when you create the bean for `OrderService` at the time of creation itself

2. Setter Injection

```
private PaymentService paymentService;
```

```
@Autowired
```

```
public void setPaymentService (PaymentService paymentService) {  
    | this.paymentService = paymentService;  
}
```

3. Field Injection (Not Recommended)

@Autowired

PaymentService paymentService;

Best Practice

- Constructor Injection is the best practice
- It allows us to declare the variable as **final**
- It allow for easier unit testing

Why?

- If we use field injection, and want to separately test that class, there is no means of providing the dependency object from the test class
- Since @Autowired sets the value of the dependent bean after its initialisation, we would not be able to use **final** access modifier for field & setter injection

Cheating Beans

PaymentService

```
package org.kdkbuilds;

import org.springframework.stereotype.Component;

@Component
public class PaymentService {

    public void pay() { 1 usage
        System.out.println("Payment done");
    }

}
```

OrderService

```
package org.kdkbuilds;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class OrderService {

    // Field Injection
    // @Autowired
    // private PaymentService paymentService;

    private final PaymentService paymentService; 2 usages
    

    // Constructor Injection
    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    // Setter Injection
    // @Autowired
    // public void setPaymentService(PaymentService paymentService) {
    //     this.paymentService = paymentService;
    // }

    public void placeOrder() { 1 usage
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Creating a Configuration class

This class is used by Spring's IOC container to feed information about bean creation & other logic

```
package org.kdkbuilds;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("org.kdkbuilds")
public class AppConfig {
    // empty for now
}
```

Creating the Application Context

```
package org.kdkbuilds;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    public static void main(String[] args) {
        // Java Reflection API
        Class<?> c1 = Student.class;
        System.out.println(c1);

        // Creating the Spring IoC Container
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);

        OrderService order = context.getBean(OrderService.class);
        order.placeOrder();
    }
}
```

Reflection API
↓

NOTE

→ If a bean uses only one constructor to wire a dependency, we can choose not to mention `@Autowired` on top of the constructor

```
private final PaymentService paymentService;  
  
// no autowired, works perfectly fine  
public OrderService (PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

Steps for Application Context

Step 1 → Start Spring's IOC Container

```
ApplicationContext context = AnnotationConfigApplicationContext (AppConfig.class);
```

Here, we are instructing Spring to create/start its IOC Container using annotation based configuration, consuming all the rules from the class metadata (`AppConfig.class`) passed.

Step 2 → Spring reads configuration from AppConfig.java

→ Which components to scan & from where

Step 3 → Spring processes @Component scanning

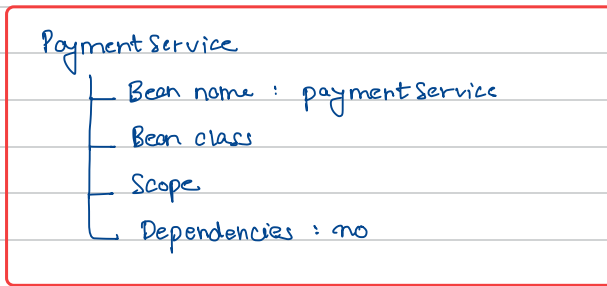
Step 4 → Spring finds @Component classes

After Spring is done mapping all the @Component files, will it directly now start creating the beans (objects) of these components?
⇒ Kind of, but there is one more step involved before it actually does so



Step 5 → Spring creates Bean definitions

Bean definition? Before actually creating the bean, it stores the metadata of all beans



Bean Definition

Why does Spring create Bean definitions? Why not directly create objects?
→ Spring's responsibility is not merely bean creation. It needs to manage them also.

Important Reasons

→ Decoupling configuration from instantiation

XML / Java @Bean / @Component $\xrightarrow{\text{abstracts}}$ standard Bean Definition s

→ Constructing complete dependency graph

Spring evaluates how all the objects depend on each other to prevent errors & manage dependency resolution.

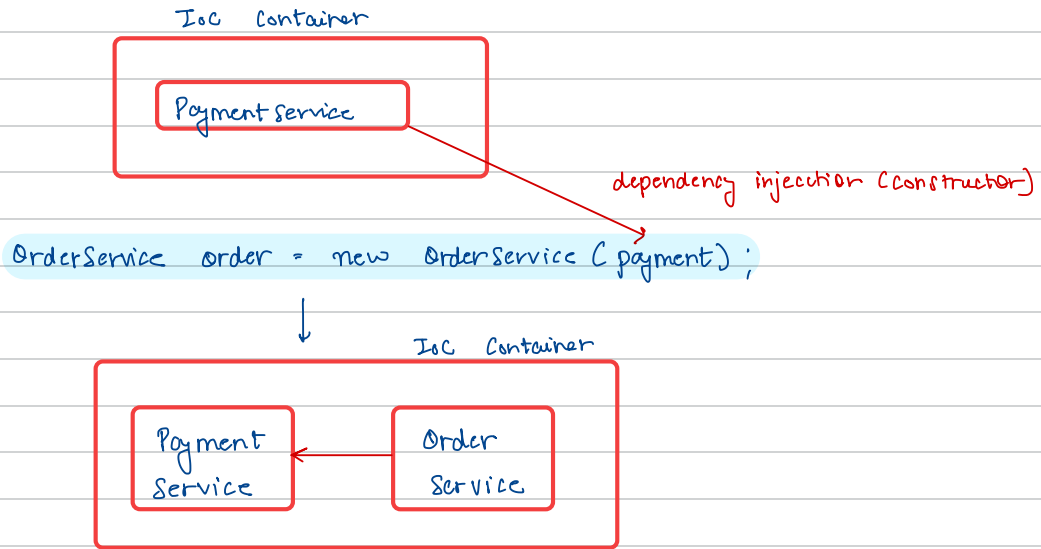
Bean Definition is an actual interface used for this purpose

Step 6 → Spring starts creating objects

Now that spring has all the Bean Definition's, it will start creating Objects

```
PaymentService payment = new PaymentService();
```

no dependencies required



Step 7 → Our application uses these beans

```
OrderService order = context.getBean(OrderService.class);  
order.placeOrder();
```

We can see that OrderService expects a bean of type PaymentService. But what if, there are multiple PaymentServices, and hence multiple PaymentService beans exist in the IoC container?

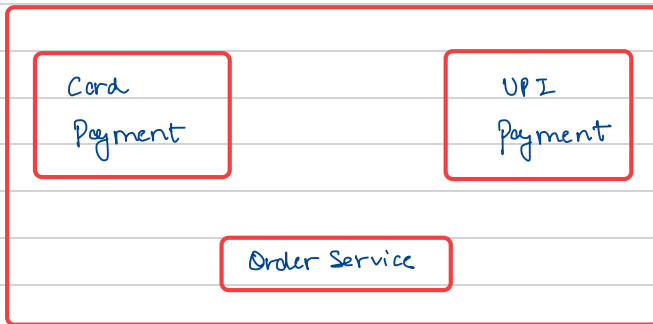
Which bean will the IoC container wire then?

Creating Loosely Coupled Systems

Let's say we have

```
org.kdkbuilds
├── payment
│   ├── PaymentService (interface)
│   ├── CardPayment (implements interface)
│   └── UPIPayment (implements interface)
├── AppConfig.java
├── OrderService.java
└── Main.java
```

Now the IoC container will look like this



which payment service bean should the IoC container wire to order service now?

→ since there is conflict present, spring will throw an error

“expected single matching bean but found 2: cardPayment, ...”

Conflict Resolution

@Primary → Placing this annotation on a component specifies spring to always use a bean of this component when conflict arises

@Qualifier ("<identifier>")

```
package org.kdkbuilds.payment;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
@Qualifier("card") ← qualifier
public class CardPayment implements PaymentService{

    @Override 1 usage
    public void pay() {
        System.out.println("Paying via Card");
    }
}
```

```
package org.kdkbuilds.payment;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
@Qualifier("upi") ← qualifier
public class UPIPayment implements PaymentService{

    @Override 1 usage
    public void pay() {
        System.out.println("Paying via UPI");
    }
}
```

mentioning qualifier during dependency injection

```
private final PaymentService paymentService; 2 usages

// Constructor Injection
@Autowired
public OrderService(@Qualifier("upi") PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

qualifier

→ We have now also loosely coupled paymentService & orderService

NOTE

→ If an explicit qualifier name is provided on the component, it uses the name of the component's class by default in camel case
→ class CardPayment → "cardPayment"

→ This approach is not recommended. Always provide appropriate qualifier names

When @Component would fail

```
public class User {
```

```
    private String name;
    private int age;
```

```
    public User (String name, int age) {
        this.name = name;
        this.age = age;
    }
```

1. Initialisation using Primitive's

```
public String getName() {  
    | return name;  
}
```

```
public void setName(String name) {  
    | this.name = name;  
}
```

```
public int getAge() {  
    | return age;  
}
```

```
public void setAge(int age) {  
    | this.age = age;  
}
```

```
}
```

Can the above class be made a bean using the @Component annotation?

→ NO, because the construction logic of the object is complicated.

→ It expects arguments, not dependencies, which Spring does not know what to provide

2. Classes of External Dependencies / JARs

→ For classes of external dependencies (that are foreign to our project) we face again a problem

→ External dependencies / JAR's code is present as .class files rather than regular java files (read only files)

→ So we can not create beans using @Component in such cases

Are we cooked then? Not really :)

Bean Annotation

Remember the config class AppConfig which was being used by Spring's application context?

→ Spring says since I can not create these objects myself, I will let the developer handle the object creation

→ Once the object is created & present inside the IoC container, I will take it forward from there & manage everything else.

→ Spring wants us to create all these objects inside the configuration class

Creating bean & User class

```
@Configuration
@ComponentScan("org.kdkbuilds")
public class AppConfig {

    @Bean ← annotation
    public User createUser() {
        return new User( age: 26, name: "Keshav");
    }
}
```

How will spring know to call the `createUser()` method & store the returned object as a bean in the IoC container?
→ Using the `@Bean` annotation

Similarly,

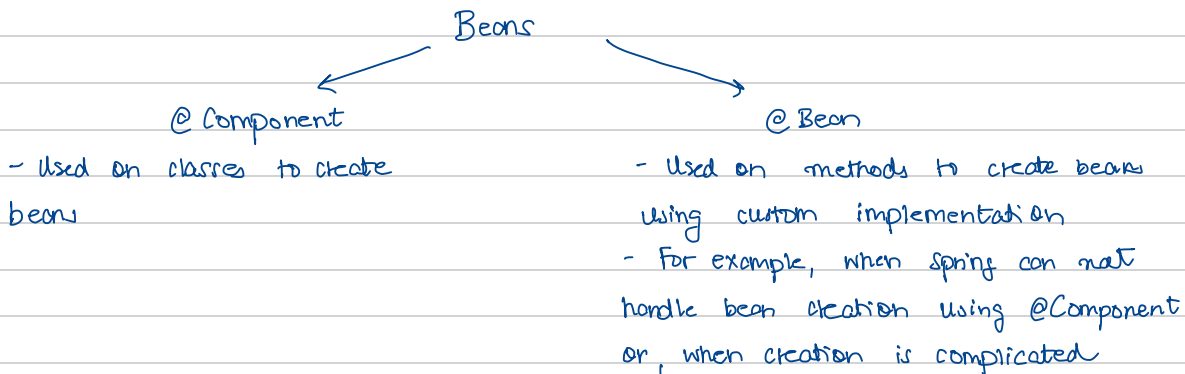
```
@Bean
public CartService getCartService() {
    return new CartService();
}
```

Hence, using application context

```
// Using beans created from custom implementation
User user = context.getBean(User.class);
System.out.println("User name: " + user.getName() + ", age: " + user.getAge());

CartService cartService = context.getBean(CartService.class);
cartService.addToCart();
```

In fact, we can also create beans of the component classes within this config class itself! `@Component` is just a convenience



Dependency Injection using @Bean Annotation

@Bean

```
public PaymentService createPaymentService() {  
    return new CardPayment();  
}
```



@Bean

```
public OrderService createOrderService(PaymentService paymentService) {  
    return new OrderService(paymentService);  
}
```

Using the above implementation inside the config class, we can create beans for native classes as well.

However, for OrderService bean, how would it receive the payment service dependency?

→ In the createOrderService() method, we have mentioned it needs PaymentService object as an argument / dependency.

→ The moment spring notices this, it checks its IoC container whether it has a PaymentService bean present.

→ As a matter of fact, it does have, as we have provided the bean creation method for it :)

→ The above implementation works very cleanly & smoothly as the dependency is being wired through a constructor injection.

→ The same could have been achieved using a setter injection as well.


```
@Bean      no dependency taken ↗
public OrderService createOrderService() {
    return new OrderService();
}
```

User Service.java



```
// Setter Injection
@Autowired ← dependency injected
public void setPaymentService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

This works perfectly fine as well, just the catch is that we will not be able to use the **final** access modifier

Conflict Resolution

- Here we are only explicitly creating CardPayment bean for Payment Service, where in fact UPIPayment exists as well.
- The moment we create a PaymentService bean for UPI, the IoC container will have more than one bean for PaymentService.
- How will spring decide which PaymentService dependency to inject into OrderService?

```

@Bean
public PaymentService createCardPaymentService() {
    return new CardPayment();
}

@Bean
public PaymentService createUpiPaymentService() {
    return new UPIPayment();
}

@Bean
public OrderService createOrderService(PaymentService paymentService) {
    return new OrderService(paymentService);
}

```



↑ This scenario will lead to conflict

→ Exactly as before using `@Primary` or `@Qualifier("<identifier>")`

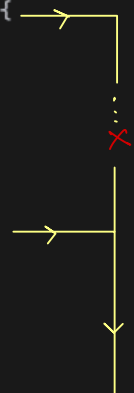
```

@Bean
public PaymentService createCardPaymentService() {
    return new CardPayment();
}

@Bean
@Primary
public PaymentService createUpiPaymentService() {
    return new UPIPayment();
}

@Bean
public OrderService createOrderService(PaymentService paymentService) {
    return new OrderService(paymentService);
}

```



```

@Bean
@Qualifier
public PaymentService createCardPaymentService() {
    return new CardPayment();
}

@Bean
@Qualifier
public PaymentService createUpiPaymentService() {
    return new UPIPayment();
}

@Bean
public OrderService createOrderService(@Qualifier("createCardPaymentService") PaymentService paymentService) {
    return new OrderService(paymentService);
}

```

→ By default, the name of the qualifier is the name of bean creation method, but we can absolutely provide our own custom identifier name as well

@Component vs @Bean, who wins?

→ Let's suppose you have created a component of the class, and defined custom bean creation logic as well.
Which will take precedence?

→ The custom @Bean configuration will take precedence, as here the developer has explicitly provided implementation details

Application Context vs BeanFactory

→ So far we know that it was ApplicationContext that was booting up the spring's IoC container

→ However, in initial Spring days, BeanFactory interface was used to boot up the IoC container

→ ApplicationContext is nothing but an additional wrapper on top of BeanFactory

